For the second clause, Trace 3 demonstrates that in a satisfied case $x_{19}, x_{22}, x_{23}, x_1$ True and x_{10} (not shown) False), positive activation occurs primarily through the set of neurons (2 and 5) that does not overlap with the activations in Trace 1 above. This indicates something impressive: the network has found feature channel coding that mostly separates clause representations. Trace 4 shows an unsatisfied case ($x_{10}, x_{19}, x_{22}, x_{23}$ True). Variable x_{10} suppresses the positive signal in neurons 2 and 5 via negative weights and activates negative witness neurons (9-16) via positive weights, leading to a False output.

As before, we could have hand constructed codes based on two monosemantic neurons, one for each of the clauses, and used large negative biases to provide a clean signal. However, the technique that gradient descent found involves coding using both positive and negative witnesses, where the negative witnesses might be partly playing the role of the bias in the final computation. In the final result, the coding is clear but uses varying weights, not clean binary values as in the hand crafted combinatorial feature channel codes presented in [1], most likely due to the pecularities of gradient descent. We believe the binary combinatorial setting may prove to be a good test ground for understanding this process better.

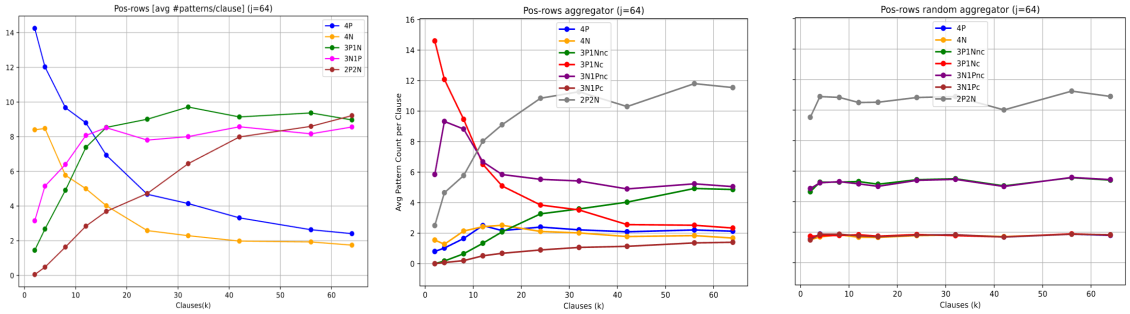

Figure 14: Left panel: a reminder of what clauses with 4 positive variables look like. Middle and right panel: Clauses with 3 positive and one negative variable. The middle panel shows the 3P1Nc coding in the positive rows that scales like the 4P pattern we saw for the 4-AND networks in the graph on the left, and very differently than the random distribution on the right.

We also analyzed the prevalence of coding patterns for this case of a DNF with three positive and one negative variable per clause as we did for networks that learn a DNF of clauses with four positive clauses. We ran the same set of tests and benchmarks. We found that the positive witnesses use a 3P1N pattern in which the negative weight in the pattern aligns with the negative variable in the clause. We will call this pattern 3P1Nc, where the c indicates "aligned with the clause," differentiating it from 3P1Nnc, the appearance of a 3P1N pattern where the negative weight is "not aligned with the clause." The left side of Figure 14 shows the 4-AND DNF positive rows coded by 4P for the case $j = 64$ that we already saw in Figure 7 (with all positive variables). The middle of Figure 14 shows the same type of plot for clauses that include a negative variable, and thus include the patterns 3P1Nc and 3P1Nnc. The right hand side shows the corresponding random pattern distribution. As can be seen, the neural network is using 3P1Nc as the clear non-random positive coding pattern for clauses with 3 positives and a negative, with a plot in red that looks very similar

to the blue 4P pattern on the right, while the 4P patterns in this case look pretty much random, as we would expect. We do not show the coding in the negative rows, but report that it uses 2P2N for lower k's but eventually saturates as k grows, looking similar to random.

5 Further Examples of Feature Channel Coding

We next take a look at other examples of problems that the network used feature channel coding to solve: ORs, clauses in CNF form, and a toy vision problem. These help further clarify this techniques versatility and generality.

5.1 OR and Conjunctive Normal Form

We here study the difference in how feature channel coding works between computing an AND and computing an OR. Within the framework of soft Boolean logic, the difference between an AND and an OR is the bias. If we ignore the magnitude of the final result, then $x_1 \wedge x_2 = \text{ReLU}(x_1 + x_2 - 1)$, while $x_1 \vee x_2 = x_1 + x_2$. Thus, we expect gradient descent to find solutions where AND has significant negative bias, whereas OR have minimal bias. To study this, we first look at a very simple Boolean formula, where the output is simply the OR of all the variables in the system. We compare this to the type of formula presented in Figure 2. We studied the same network as in that scenario (hidden dimension equals input dimension equals 16, single neuron at the second layer), and we examined the Layer 1 bias for the neurons at layer 1 with positive Layer 2 weights associated with them.

	Average layer 1 bias	Maximum absolute layer 1 bias
AND	-0.61	1.54
OR	0.0079	0.0306

Table 1: Comparison of AND vs OR layer 1 bias for neurons with positive layer 2 weight.

Table 1 summarizes the result of 10 random network trainings, each consisting of 10000 randomly generated inputs (settings of the variables). For AND, the inputs are as described for Figure 2. For OR, each input is a 1 independently with probability 0.043 and 0 otherwise; this provides approximately equal probability of a positive and negative instance. In this table, we see that the result are as expected: the AND function has a negative bias, which on average is large enough to have an impact, although as already discussed, the value of the bias tends to be a bit smaller than the theory would predict. The OR function, on the other hand, has a very small bias, with a negligible impact on the compuatation.

In an effort to further study the OR function, a seemingly good test bed would be to study formulas in Conjunctive Normal Form (CNF), which is the AND of a number of clauses, where each clause is the OR of a number of literals. The network solves the DNF formulas studied above by computing the AND clauses at the first layer, and then using the single neuron at the second layer to compute an OR of those results, and so our expectation was that the network would solve a CNF formula by computing the OR clauses at the first layer, and then use the second layer to compute the AND of those results. Our expectation was wrong, but we actually found something more interesting.

To study CNF, our neural network setup is the same as the above, with input dimension 16 and hidden dimension 16. We study CNF formulas where every clause is the OR of two (positive) variables, and each variable is used exactly once (the *feature influence* [1] of the network is 1). To train any given formula, we want to construct a roughly even split between True inputs and False inputs; we do this simply by choosing every variable to be True independently with probability 0.75. We studied a number of CNF formulas chosen randomly from the set of such formulas, and saw very consistent results. We here present results for one such randomly chosen formula:

$$(x_3 \vee x_{11}) \wedge (x_9 \vee x_{12}) \wedge (x_2 \vee x_8) \wedge (x_0 \vee x_{10}) \wedge (x_{13} \vee x_{14}) \wedge (x_4 \vee x_7) \wedge (x_1 \vee x_5) \wedge (x_6 \vee x_{15}).$$

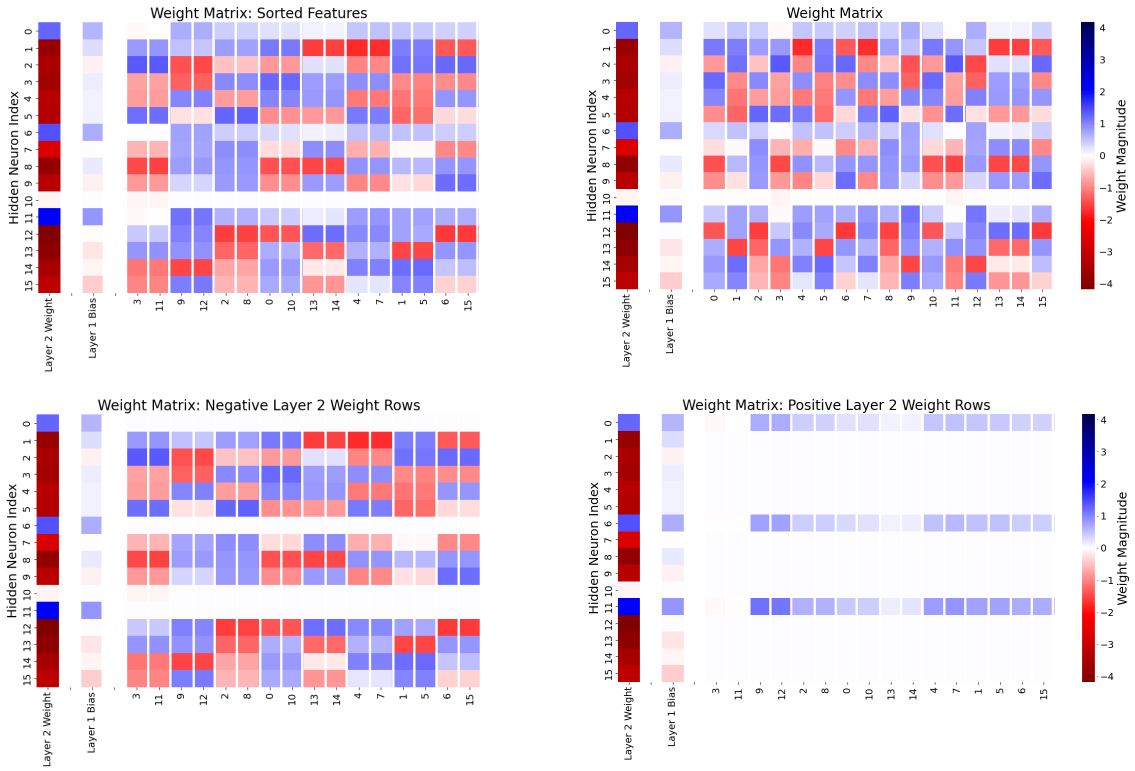

Figure 15: All weights and biases for a neural network trained on the Boolean formula $(x_3 \vee x_{11}) \wedge (x_9 \vee x_{12}) \wedge (x_2 \vee x_8) \wedge (x_0 \vee x_{10}) \wedge (x_{13} \vee x_{14}) \wedge (x_4 \vee x_7) \wedge (x_1 \vee x_5) \wedge (x_6 \vee x_{15})$

In Figure 15, we depict what the network learns for this formula with 40,000 randomly chosen inputs. We see in this figure that the network is not learning the AND as we had expected, and in fact, there is not much that is interesting happening on the small number of positive layer 2 weight rows at all. On the other hand, the negative rows look much more interesting, and they take up the majority of the neurons. Furthermore, those rows do fit the pattern of feature channel coding, but